



中山大學
SUN YAT-SEN UNIVERSITY

运算符重载 (上)

中山大学计算机学院



主讲人：万海



wanhai@mail.sysu.edu.cn

目录

CONTENTS

01

运算符重载概念

02

普通运算符重载

03

自增/自减运算符重载

04

赋值运算符重载

05

浅复制与深复制策略

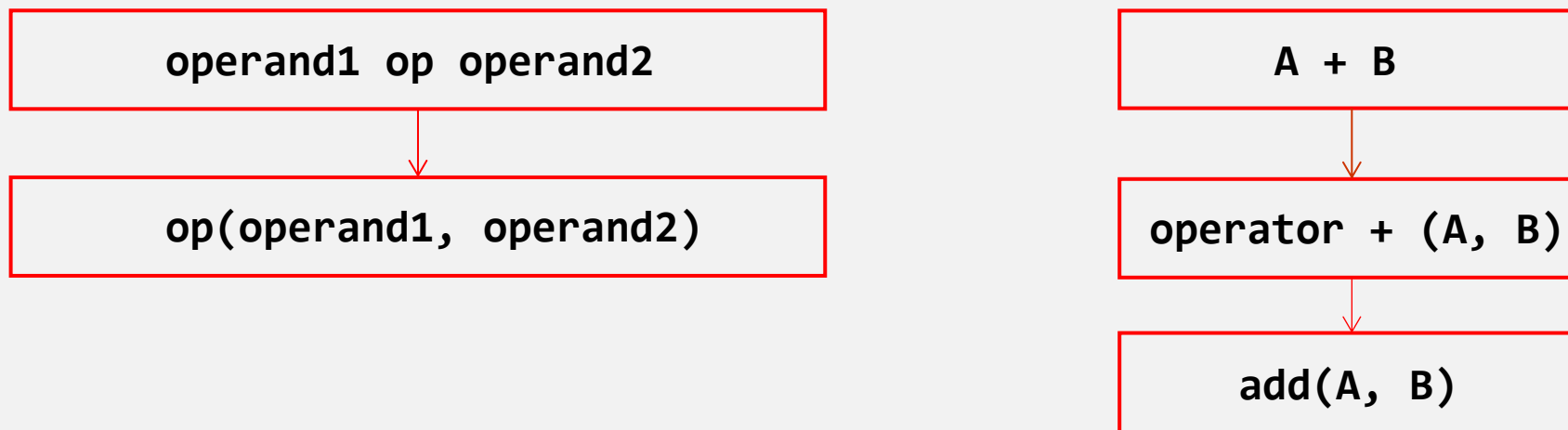


运算符重载

不用类型的数据使用习惯的运算符是数学表达式的习惯，如复数、向量都有 + 和 * 等运算。

C++使用特别函数名的函数重载技术，根据实参类型，使得运算符可处理不同类型数据。

重载运算符可以理解为特殊名称的函数，函数名由关键字 operator 和 运算符构成。





运算符重载的方法

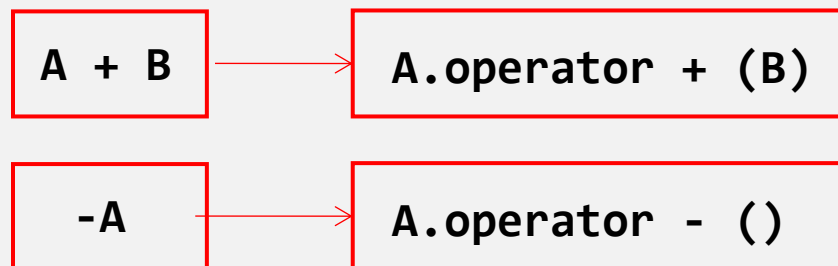
运算符重载的两种方法：

- 类成员函数运算符重载（类内定义）

- `return_type class_name::operator op(operand2) {}`
- 重载二元运算符时，成员运算符函数只需显式传递一个参数（即二元运算符的右操作数），而左操作数则是该类对象本身，通过 `this` 指针隐式传递。
- 重载一元运算符时，成员运算符函数没有参数，操作数是该类对象本身，通过 `this` 指针隐式传递

- 友元函数运算符重载（类外定义）

- `return_type operator op(operand1, operand2) {}`
- 后面会详细说明



可以重载的运算符

	运算符	操作符
二元	算术运算符	+ (加), -(减), *(乘), /(除), %(取模)
	关系运算符	==(等于), != (不等于), < (小于), > (大于), <=(小于等于), >=(大于等于)
	逻辑运算符	(逻辑或), &&(逻辑与), !(逻辑非)
	位运算符	(按位或), & (按位与), ~(按位取反), ^(按位异或), << (左移), >>(右移)
	单目运算符	+ (正), -(负), *(指针), &(取地址)
	自增自减运算符	++(自增), --(自减)
	赋值运算符	=, +=, -=, *=, /=, %=, &=, =, ^=, <<=, >>=
	其他运算符	()(函数调用), -(成员访问), (逗号), [] (下标)
(超纲)	空间申请与释放	new, delete, new[], []delete



不可以重载的运算符

:: (作用域解析)

. (成员访问)

.* (通过成员指针的成员访问)

?: (三元条件)

sizeof: 运算符 (包括除 **new**, **delete** 外的关键字运算符, 如 **alignof**, **typeid** 等等)

#: 预处理符号

其他限制:

- 不能创建新运算符, 例如 ******、**<>** 或 **&|**。
- 运算符 **&&** 与 **||** 的重载失去短路求值。
- 重载的运算符 **->** 必须要么返回裸指针, 要么 (按引用或值) 返回同样重载了运算符 **->** 的对象。
- 不可能更改运算符的优先级、结合方向或操作数的数量。



双目，单目运算符重载

单、双目运算符包括算术、比较、逻辑、逐位等。其中算术运算符的**内建原型**声明如下：

运算符名	语法	可重载	原型示例 (对于 <code>class T</code>)	
			类内定义	类外定义
一元加	<code>+a</code>	是	<code>T T::operator+() const;</code>	<code>T operator+(const T &a);</code>
一元减	<code>-a</code>	是	<code>T T::operator-() const;</code>	<code>T operator-(const T &a);</code>
加法	<code>a + b</code>	是	<code>T T::operator+(const T2 &b) const;</code>	<code>T operator+(const T &a, const T2 &b);</code>
减法	<code>a - b</code>	是	<code>T T::operator-(const T2 &b) const;</code>	<code>T operator-(const T &a, const T2 &b);</code>
乘法	<code>a * b</code>	是	<code>T T::operator*(const T2 &b) const;</code>	<code>T operator*(const T &a, const T2 &b);</code>
除法	<code>a / b</code>	是	<code>T T::operator/(const T2 &b) const;</code>	<code>T operator/(const T &a, const T2 &b);</code>
模	<code>a % b</code>	是	<code>T T::operator%(const T2 &b) const;</code>	<code>T operator%(const T &a, const T2 &b);</code>

1. 查阅手册，一般情况用户定义运算符应能与**内建版本相同的方式使用**。即使用内建声明
2. 部分情况下，需要**返回左值对象**，可使用 `T&` 作为返回值。例如 `operator<<` 和 `operator>>` 作为流插入和流提取的重载所返回的是 `T&`（左操作数的引用即 `cout` 或 `cin`，以实现连续输出或输入）。
3. `T2` 可以是包括 `T` 在内的任何类型。



双目，单目运算符 - 成员函数重载例子

```
class Integer {  
    int x;  
public:  
    Integer(int x=0):x(x)    {  
    }  
    Integer operator + (const Integer& Int) {  
        return Integer(x+Int.x);  
    }  
    Integer operator - (const Integer& Int) {  
        return Integer(x-Int.x);  
    }  
    Integer operator - ()    {  
        return Integer(-x);  
    }  
    void print()    {  
        cout<<x<<endl;  
    }  
};
```

```
int main() {  
    Integer a=3, b=4;  
    a.print();  
    b.print();  
    Integer c=a+b; // same as a.operator+(b)  
    c.print();  
    Integer d=a-b;  
    d.print();  
    Integer e=-a;  
    e.print();  
    return 0;  
}
```

输出结果:

3
4
7
-1
-3

注意（参见例子中问题）：

第一（左）操作数：必须是 *this;

第二（右）操作数：类型任意，注意区分 T, **const T&**, T&, T&&, T*, const T* 的差别;

返回值：可以是任意类型，一般是左操作数类型的临时对象T或左操作数自身的引用T&。



特殊运算符：自增运算符

重载自增运算符时需注意：

- 若为前缀自增运算符，直接重载（以++举例）：
 - `return_type class_name::operator ++()` {}
- 若为后缀自增运算符，该函数有一个 `int` 类型的虚拟形参，这个形参在函数的主体中是不会被使用的，这只是一个约定，它告诉编译器递增运算符正在**后缀模式**下被重载：
 - `return_type class_name::operator ++(int)` {}

运算符名	语法	可重载	原型示例（对于类 <code>class T</code> ）	
			类内定义	类外定义
前自增	<code>++a</code>	是	<code>T& T::operator++();</code>	<code>T& operator++(T& a);</code>
前自减	<code>--a</code>	是	<code>T& T::operator--();</code>	<code>T& operator--(T& a);</code>
后自增	<code>a++</code>	是	<code>T T::operator++(int);</code>	<code>T operator++(T& a, int);</code>
后自减	<code>a--</code>	是	<code>T T::operator--(int);</code>	<code>T operator--(T& a, int);</code>
注解 • 内建运算符的前置版本返回引用而后置版本返回值，典型的用户定义重载也遵循该模式，从而使用户定义运算符可以与内建版本相同的方式使用。然而，用户定义重载中，能以任何类型为返回类型（包括 <code>void</code>）。 • <code>int</code> 形参是用于区别运算符的前置和后置版本的空设形参。调用用户定义的运算符时，该形参中传递的值始终为零，但通过函数调用记法调用这个运算符时可以更改它（例如 <code>a.operator++(2)</code> 或 <code>operator++(a, 2)</code>）。				



自增运算符

```
class Integer {  
    int x;  
public:  
    Integer(int x=0):x(x) {  
    }  
    Integer& operator ++ () {  
        cerr<<"prefix is invoked"<<endl;  
        ++x;  
        return *this;  
    }  
    Integer operator ++ (int) {  
        cerr<<"suffix is invoked"<<endl;  
        return Integer(x++);  
    }  
    void print() {  
        cout<<x<<endl;  
    }  
};
```

```
int main() {  
    Integer a=3;  
    a.print();  
    Integer b=++a;  
    a.print();  
    b.print();  
    Integer c=a++;  
    a.print();  
    c.print();  
    return 0;  
}
```

输出结果:

```
3  
prefix is invoked  
4  
4  
suffix is invoked  
5  
4
```

要点:

1. 内建前缀运算符与后缀运算符原型不一样
2. 前缀运算符, 先增后用, 一般返回增后的对象的引用 (左值)
3. 后缀运算符, 先用后增, 先建立原始对象的副本再自增, 一般返回副本对象 (右值)



特殊运算符：赋值运算符

赋值运算符的内建原型声明如下：

运算符名	语法	可重载	原型示例 (对于 <code>class T</code>)	
			类内定义	类外定义
简单赋值	<code>a = b</code>	是	<code>T& T::operator =(const T2& b);</code>	N/A
加法赋值	<code>a += b</code>	是	<code>T& T::operator +=(const T2& b);</code>	<code>T& operator +=(T& a, const T2& b);</code>
减法赋值	<code>a -= b</code>	是	<code>T& T::operator -=(const T2& b);</code>	<code>T& operator -=(T& a, const T2& b);</code>
乘法赋值	<code>a *= b</code>	是	<code>T& T::operator *=(const T2& b);</code>	<code>T& operator *=(T& a, const T2& b);</code>
除法赋值	<code>a /= b</code>	是	<code>T& T::operator /=(const T2& b);</code>	<code>T& operator /=(T& a, const T2& b);</code>
取模赋值	<code>a %= b</code>	是	<code>T& T::operator %=(const T2& b);</code>	<code>T& operator %=(T& a, const T2& b);</code>
逐位与赋值	<code>a &= b</code>	是	<code>T& T::operator &=(const T2& b);</code>	<code>T& operator &=(T& a, const T2& b);</code>
逐位或赋值	<code>a = b</code>	是	<code>T& T::operator =(const T2& b);</code>	<code>T& operator =(T& a, const T2& b);</code>
逐位异或赋值	<code>a ^= b</code>	是	<code>T& T::operator ^=(const T2& b);</code>	<code>T& operator ^=(T& a, const T2& b);</code>
逐位左移赋值	<code>a <<= b</code>	是	<code>T& T::operator <<=(const T2& b);</code>	<code>T& operator <<=(T& a, const T2& b);</code>
逐位右移赋值	<code>a >>= b</code>	是	<code>T& T::operator >>=(const T2& b);</code>	<code>T& operator >>=(T& a, const T2& b);</code>
注意 - 所有内建赋值运算符都返回 <code>*this</code>，而大多数用户定义重载也会返回 <code>*this</code>，从而能以与内建版本相同的方式使用用户定义运算符。然而，用户定义运算符重载中，返回类型可以是任意类型（包括 <code>void</code>）。 - T2 可以是包含 T 在内的任意类型				



特殊运算符：简单赋值运算符

C++允许重载赋值运算符：

```
class Integer {  
    int x;  
public:  
    Integer(int x=0):x(x)    {  
    }  
    Integer& operator = (const Integer& Int)    {  
        x=Int.x;  
        cout<<"function is invoked"<<endl;  
        return *this;  
    }  
    void print()    {  
        cout<<x<<endl;  
    }  
};  
  
int main() {  
    Integer a=3, b;  
    a.print();  
    b=a;  
    b.print();  
    return 0;  
}
```

输出结果：

3

function is invoked

3



赋值运算符：运算赋值

注意简单赋值和运算赋值之间的区别

```
class Integer {
    int x;
public:
    Integer(int x=0):x(x) {
        cout<<"constructing Int "<< x <<endl;
    }
    Integer& operator = (const Integer& other) {
        if (this != &other) {
            cout<<"operator= invoked"<<endl;
            x=other.x;
        }
        return *this;
    }
    Integer& operator += (const Integer& other) {
        cout<<"operator+= invoked"<<endl;
        //this->x += other.x;    //[1]
        *this = this->operator +(other);//[2]
        return *this;
    }
    Integer operator + (const Integer& Int) {
        cout<<"operator+ invoked"<<endl;
        return Integer(x+Int.x);
    }
}
```

```
void print() {
    cout<<x<<endl;
}

};

int main() {
    Integer a=3, b;
    b=5;
    b=a;
    b+=b;
    return 0;
}
```

分析:

1. 解释程序的输出结果
2. 注意 Integer a=3; 和 b=5 的区别
3. 注释去除op+=, b+=b会执行先加再赋值吗?
4. 研究+=函数, 语句[1]和[2]可以互换, 你喜欢用哪一个? 各有哪些好处?

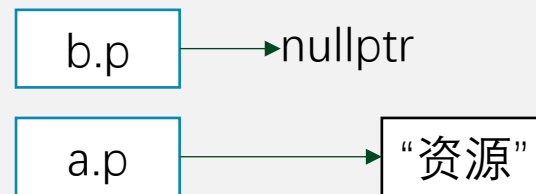
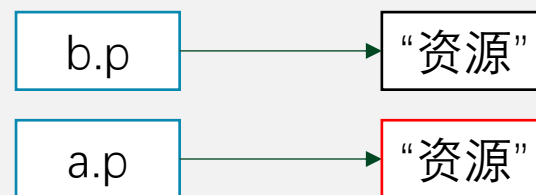
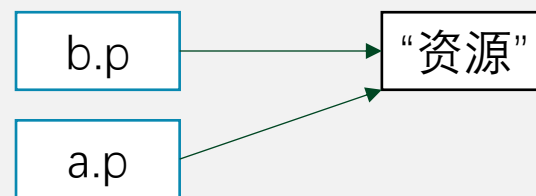


赋值运算符：常见语义

利用赋值运算符重载特性，可实现多种语义的赋值，常见的：

- 浅拷贝赋值
 - 由缺省的赋值运算符完成，会按成员声明顺序逐一调用成员的赋值运算
 - 当成员是指针时，被赋值对象与赋值对象共用一个资源
- 深拷贝赋值
 - 由用户定义的赋值运算符完成。该函数不需要逐一调用非指针成员的赋值运算（但建议按声明顺序逐一赋值）
 - 当成员是指针时，需要释放已拥有资源，复制赋值对象对应的资源，并指向该资源的副本，即被赋值对象拥有赋值对象资源的副本
- 移动赋值
 - 由自定义的 `operator=(T&&)` 赋值运算符完成。默认赋值运算被弃置。
 - 当成员是指针时，需要释放已拥有资源，并指向赋值对象对应的资源，同时赋值相关成员。赋值对象指针赋值空，并被禁用。即赋值对象资源转移入被赋值对象

`T a; a = b;`





赋值运算符：浅拷贝策略问题分析

所以当成员变量包含指针类型的时候，需要注意浅拷贝潜在的问题：

```
class IntArray {
    int *a;
    int n;
public:
    IntArray(int n=1):n(n) {
        a=new int[n];
        for (int i=0;i<n;++i) a[i]=i;
    }
    ~IntArray() {
        cout << "release p=" << a <<endl;
        delete[] a;
    }
    void print() {
        for (int i=0;i<n;++i)
            cout<<a[i]<<" ";
        cout<<endl;
    }
};
```

```
int main() {
    IntArray a(4), b;
    a.print();
    b=a; // default operator = is ok, but ...
    b.print();
    a.print(); //a,b point to the same object
    return 0;
}
```

1.内存泄漏
2. b, a先后析构，资源二次释放
内存段错误

```
0 1 2 3
0 1 2 3
0 1 2 3
release p=0xb615c0
release p=0xb615c0
```

Process exited after 32.67 seconds with return value 3221226356
请按任意键继续...



赋值运算符：深拷贝策略

所以当成员变量包含指针类型的时候，一般需要使用深拷贝策略：

```
class IntArray {
    int *a;
    int n;
public:
    IntArray(int n=1):n(n) {
        a=new int[n];
        for (int i=0;i<n;++i) a[i]=i;
    }
    ~IntArray() {
        cout << "release p=" << a <<endl;
        delete[] a;
    }
    IntArray& operator = (const IntArray& other) {
        if (this != &other) {
            delete[] a; //这里a一定不为空
            n=other.n;
            a=new int[n];
            memcpy(a,other.a,sizeof(int)*n);
        }
        return *this;
    }
    void print() {
        for (int i=0;i<n;++i) cout<<a[i]<<" ";
        cout<<endl;
    }
};
```

```
int main() {
    IntArray a(4), b;
    a.print();
    b=a; // 调用深拷贝策略赋值
    b.print();
    a.print(); //a,b point to ...
    return 0;
}
```

1.b中资源先释放
再指向a资源的副本
2.b, a各自释放资源

```
0 1 2 3
0 1 2 3
0 1 2 3
release p=0x8215e0
release p=0x8215c0
```

Process exited after 0.06927 seconds with return value 0
请按任意键继续...



赋值运算符：深拷贝策略（官方指南）

赋值运算符（`operator=`）有特殊性质：细节见[复制赋值与移动赋值](#)。

对规范的复制赋值运算符，期待其在[自赋值时不进行操作](#) , 并按引用返回 lhs:

```
// 假设对象保有可重用存储，例如一个分配于堆的缓冲区 mArray
T& operator=(const T& other) // 复制赋值
{
    if (this != &other) { // 期待检查自赋值
        if (other.size != size) { // 存储不可复用
            delete[] mArray; // 销毁 this 中的存储
            size = 0;
            mArray = nullptr; // 在下一行抛出的情况下维持不变式
            mArray = new int[other.size]; // 分配 this 中的存储
            size = other.size;
        }
        std::copy(other.mArray, other.mArray + other.size, mArray);
    }
    return *this;
}
```



赋值运算符：移动策略₁【超纲】

预备知识(1): 类型 T, const T&, T&, T&& 作为形参的区别

- T 接受 T类型左值 或 T2类型右值 的实参, 并构建新对象
- const T& 接受 T类型左值 或 T2类型右值 的实参, 函数不修改引用
- T& 仅接受 T类型左值的实参, 可修改函数外部对象
- T&& 仅接受 T2类型右值 的实参, 可修改函数外部对象
- 其中: T2 与 T兼容 或 存在构造函数 T(T2), T(const T2&) 或 T(T2&&)

预备知识(2): 强制转为右值

- 在标头 <utility> 定义的 **std::move**(T&& a) 函数将 a 转型为右值引用



赋值运算符：移动策略₂【超纲】

运用移动策略可确保函数外、函数内申请的资源在合适的位置正确释放！

```
class IntArray {
    int *a;
    int n;
public:
    IntArray(int n=1):n(n) {
        cout << "constructing.. p=";
        a=new int[n];
        for (int i=0;i<n;++i) a[i]=i;
        cout << a <<endl;
    }
    IntArray(IntArray&& other) { //移动构造
        cout << "copy move... p=" << other.a <<endl;
        a = other.a;n = other.n;
        other.a = nullptr; other.n=0;
    }
    ~IntArray() {
        cout << "release p=" << a <<endl;
        if (a) delete[] a;
    }
}
```

```
IntArray& operator = (IntArray&& other) { //移动赋值
    cout << "operator = move p=" << other.a <<endl;
    if (this != &other) {
        if (other.a) delete[] a;
        a = other.a;n = other.n;
        other.a = nullptr; other.n=0;
    }
    return *this;
}
IntArray& inc() {
    for (int i=0;i<n;++i) a[i]++;
    return *this;
}
void print() {
    for (int i=0;i<n;++i) cout<<a[i]<<" ";
    cout<<endl;
}
};
```



赋值运算符：移动策略₃【超纲】

运用移动策略可确保函数外，函数内申请的资源在合适的位置正确释放！

```
IntArray foo(IntArray a) {  
    a.inc();  
    a.inc();  
    return a;  
}  
  
IntArray bar() {  
    IntArray a(8);  
    a.inc();  
    return a;  
}
```

请自己分析每个部分的结果哦！

```
int main() {  
    IntArray a(4), b, c;  
    a.print();  
    b=std::move(a); //a看作右值，调用移动赋值  
    a.print(); //a nothing  
    b.print();  
    cout<< "1-----"<<endl;  
  
    c = foo(std::move(b));  
    a.print();  
    b.print();  
    c.print();  
    cout<< "2-----"<<endl;  
  
    IntArray d; //d=bar(); //会更高效！  
    d=std::move(bar());  
    d.print();  
    cout<< "3-----"<<endl;  
  
    return 0;  
}
```



赋值运算符：移动策略₄【超纲】

运用移动策略可确保函数外，函数内申请的资源在合适的位置正确释放！

```
constructing.. p=0xae15a0
constructing.. p=0xae15c0
constructing.. p=0xae15e0
0 1 2 3
operator = move p=0xae15a0

0 1 2 3
1-----
copy move... p=0xae15a0
copy move... p=0xae15a0
operator = move p=0xae15a0
release p=0
release p=0

2 3 4 5
2-----
```

```
constructing.. p=0xae15c0
constructing.. p=0xae1aa0
operator = move p=0xae1aa0
release p=0
1 2 3 4 5 6 7 8
3-----
release p=0xae1aa0
release p=0xae15a0
release p=0
release p=0

-----
Process exited after 0.06519 seconds with return value 0
```



练习：Vector3 (3)

- 运算符

<u>operator -</u>	将一个向量减去另一个向量。
<u>operator !=</u>	Returns true if vectors are different.
<u>operator *</u>	将向量乘以一个数值。
<u>operator /</u>	将向量除以一个数值。
<u>operator +</u>	将两个向量相加。
<u>operator ==</u>	如果两个向量大致相等，则返回 true。

? 本次课后仅需完成 $A*c$ 不用完成 $c*A$ 。为什么 $c*A$ 无法实现呢？



练习：Vector3 (3)

请完成以下任务：

- 定义文件 `vector3.hpp`
- 给出成员函数实现 `vector3.cpp`
- 编写客户端程序 `point.cpp`
 - 构造合适例子验证运算符实现的正确性



中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院



主讲人：XXX



中山大学MOOC课程组